Campus Parking Lot Tracker — Database Design

CS 514 Database Theory and Implementation · Team project · Database design (rudimentary, v1) · Author: Vishnu

1. Introduction and scope

This document describes the relational database behind the Parking Lot Tracker. The database stores parking-lot facts and time-stamped occupancy readings so that the website and the analysis scripts can answer: *how full is each lot, when is it open or closed, what special requirements apply, and when is parking free?*

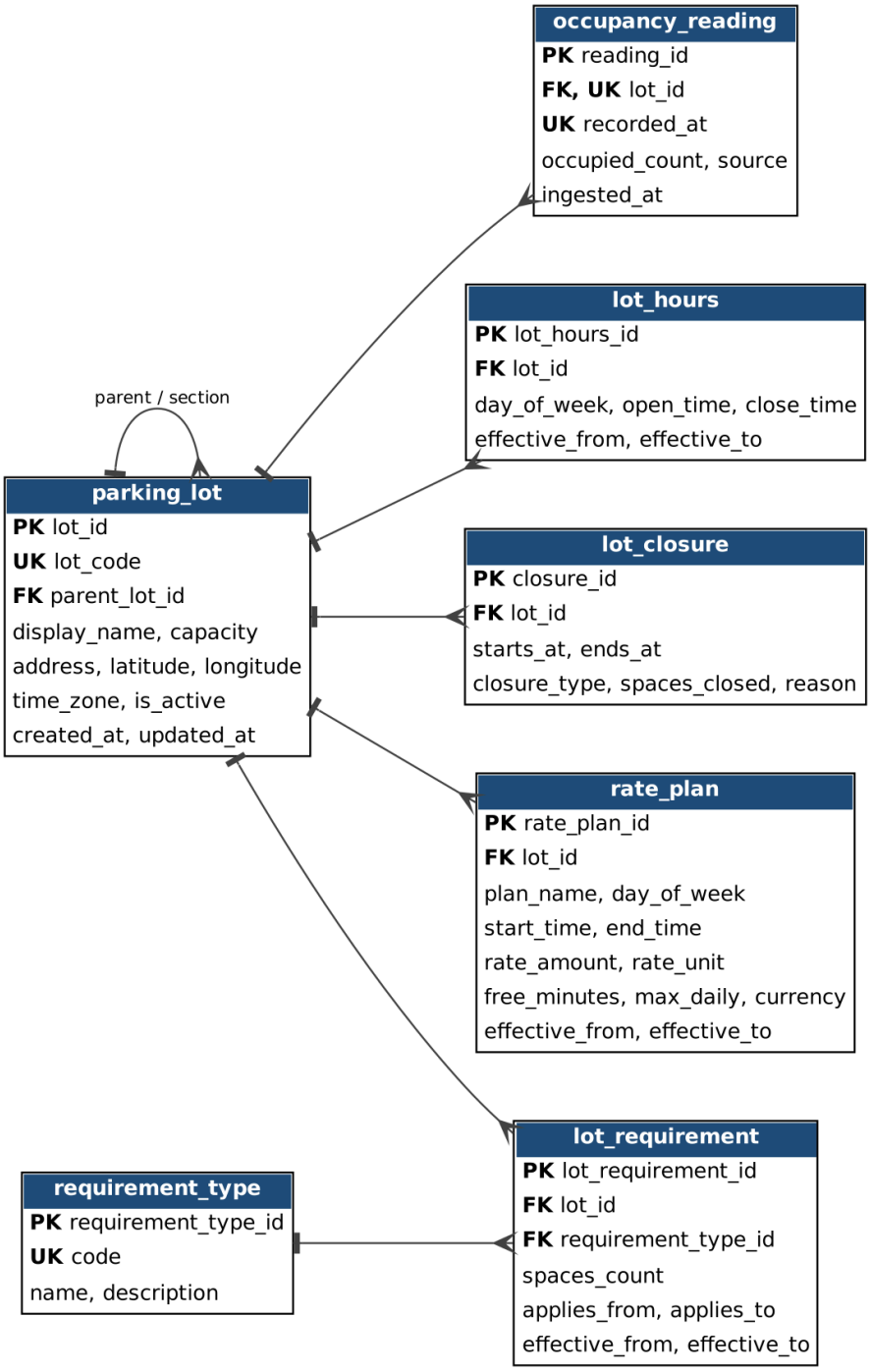
The design is implemented in **MySQL 8.0** (DDL in `schema.sql`). The only real data is the UCI *Parking Birmingham* dataset (ID 482, CC BY 4.0: 35,717 rows, 30 lots, Oct–Dec 2016). The UCI data has no hours, closures, requirements or prices. Those tables are designed and constrained now but are **empty** until the team supplies data; any sample values added later will be synthetic and labelled as such. The repository contains no credentials or personal information.

2. Source data mapping

UCI CSV column	Database column	Handling
SystemCodeNumber	parking_lot.lot_code	One lot row per distinct code (30)
Capacity	parking_lot.capacity	Constant per lot in the source, so stored once on the lot
Occupancy	occupancy_reading.occupied_count	Kept as-is (signed) and flagged in the view
LastUpdated	occupancy_reading.recorded_at	Parsed to DATETIME; unique per lot

Findings in the source: 216 exact duplicate rows (dropped by the unique key), 12 negative counts, 373 counts above capacity, and a lot hierarchy: `NIA Car Parks` equals `NIA North` + `NIA South` (capacities 480 + 788 = 1268; occupancy sums match). That hierarchy is modelled with `parent_lot_id`.

3. Entity-relationship diagram



Crow's-foot notation. Every child table has a mandatory parent (`parking_lot`) with zero or more child rows; `lot_requirement` is the associative table for the many-to-many between lots and requirement types; `parking_lot` has an optional self-reference (a section has at most one parent lot).

4. Glossary

Name	Synonyms	Description
Parking lot	Car park; facility; system code	A named place where vehicles park. Stored in parking_lot.
Section	Sub-lot; zone	A lot that belongs to a larger lot (NIA North and NIA South inside NIA Car Parks). Linked by parent_lot_id so totals are not double-counted.
Capacity	Total spaces; bays	Maximum number of vehicles a lot is designed to hold.
Occupancy reading	Observation; snapshot; sensor update	One measurement of how many spaces were taken at one moment (a CSV row).
Occupied count	Occupancy; cars parked	Number of spaces in use at the time of a reading.
Occupancy rate	Utilisation; fullness	occupied_count ÷ capacity, computed in the view v_occupancy.
Free spaces	Availability; vacancies	capacity − occupied_count; derived, not stored.
Quality flag	Data-quality status	NEGATIVE, OVER_CAPACITY or OK. Marks anomalous raw readings without deleting them.
Opening hours	Operating hours; schedule	Regular weekly times a lot is open (lot_hours).
Closure	Outage; restriction; shutdown	A one-off full or partial closure that overrides opening hours (lot_closure).
Requirement	Access rule; permit rule; restriction	A condition for using a lot or some of its spaces, such as a disability permit, faculty-only or paid.
Rate plan	Tariff; pricing; fee schedule	Price for a lot during a day and time band.
Free hours	Free period; grace period	A time band with rate_amount 0, or the free_minutes before charging starts.
Effective window	Validity period	effective_from / effective_to dates during which a rule applies; NULL end = still in force.

5. Table definitions

Conventions: surrogate integer primary keys, natural keys kept as UNIQUE; all DATETIME/TIME values are lot-local wall-clock time (`parking_lot.time_zone`); ISO weekday numbers; `effective_to` NULL means still in force; CHECK constraints need MySQL 8.0.16+.

`parking_lot`

One row per lot, or per section of a lot (self-referencing parent).

Column	Type	Key / constraint	Notes
lot_id	INT UNSIGNED	PK, auto-increment	Surrogate key
lot_code	VARCHAR(32)	UNIQUE, NOT NULL	Natural key; CSV SystemCodeNumber

display_name	VARCHAR(100)	NULL	Friendly name for the website
parent_lot_id	INT UNSIGNED	FK → parking_lot, ON DELETE SET NULL	Containing lot (e.g. NIA North → NIA Car Parks)
capacity	INT UNSIGNED	NOT NULL, CHECK > 0	Total spaces; CSV Capacity
address / latitude / longitude	VARCHAR / DECIMAL(9,6)	NULL, CHECK lat ±90, lon ±180	Location (not in UCI data)
time_zone	VARCHAR(64)	NOT NULL, default 'Europe/London'	Zone in which all lot times are expressed
is_active	BOOLEAN	NOT NULL, default TRUE	Soft retire a lot
created_at / updated_at	TIMESTAMP	NOT NULL, defaults	Audit columns

`occupancy_reading`

Time series of occupied-space counts (the UCI rows).

Column	Type	Key / constraint	Notes
reading_id	BIGINT UNSIGNED	PK, auto-increment	Surrogate key
lot_id	INT UNSIGNED	FK → parking_lot, ON DELETE CASCADE	Lot measured
recorded_at	DATETIME	NOT NULL	CSV LastUpdated (lot-local time)
occupied_count	INT (signed)	NOT NULL	CSV Occupancy; signed, no upper bound so raw anomalies are kept
source	VARCHAR(32)	NOT NULL, default 'UCI_BHAM_2016'	Origin of the row; lets live feeds coexist with the dataset
ingested_at	TIMESTAMP	NOT NULL, default now	Load time
(lot_id, recorded_at)	—	UNIQUE	Business key; also serves lot+time-range queries

`lot_hours`

Regular weekly opening hours.

Column	Type	Key / constraint	Notes
lot_hours_id	INT UNSIGNED	PK	Surrogate key
lot_id	INT UNSIGNED	FK, ON DELETE CASCADE	Lot
day_of_week	TINYINT UNSIGNED	CHECK 1–7	ISO: 1 = Monday … 7 = Sunday
open_time / close_time	TIME	NOT NULL	close ≤ open means past midnight; 00:00/00:00 means 24 h
effective_from / effective_to	DATE	from NOT NULL; to NULL = open-ended; CHECK to ≥ from	Validity window, so schedule changes keep history

(lot_id, day_of_week, effective_from, open_time)	—	UNIQUE	Several rows per day allowed (split hours); no row = closed that day

`lot_closure`

One-off closures that override regular hours.

Column	Type	Key / constraint	Notes
closure_id	INT UNSIGNED	PK	Surrogate key
lot_id	INT UNSIGNED	FK, ON DELETE CASCADE	Lot
starts_at / ends_at	DATETIME	ends_at NULL = until further notice; CHECK ends > starts	Closure window
closure_type	ENUM('FULL','PARTIAL')	NOT NULL, default FULL	Whole lot or part of it
spaces_closed	INT UNSIGNED	CHECK: NULL if FULL; > 0 if PARTIAL	Number of spaces unavailable
reason	VARCHAR(255)	NULL	Event, maintenance, weather …

`requirement_type`

Lookup of access rules (extensible by inserting rows).

Column	Type	Key / constraint	Notes
requirement_type_id	SMALLINT UNSIGNED	PK	Surrogate key
code	VARCHAR(32)	UNIQUE, NOT NULL	Stable identifier, e.g. DISABILITY_PERMIT
name / description	VARCHAR	name NOT NULL	Display text
Seed rows	—	—	DISABILITY_PERMIT, FACULTY_ONLY, PAID

`lot_requirement`

Resolves the many-to-many between lots and requirement types.

Column	Type	Key / constraint	Notes
lot_requirement_id	INT UNSIGNED	PK	Surrogate key
lot_id	INT UNSIGNED	FK, ON DELETE CASCADE	Lot
requirement_type_id	SMALLINT UNSIGNED	FK, ON DELETE RESTRICT	Rule that applies
spaces_count	INT UNSIGNED	NULL = whole lot; CHECK > 0	How many spaces the rule covers
applies_from / applies_to	TIME	both NULL or both set (CHECK)	Daily window; NULL = all day
effective_from / effective_to	DATE	CHECK to ≥ from	Validity window

`rate_plan`

Pricing per lot, day and time band; free hours are rate_amount = 0.

Column	Type	Key / constraint	Notes
rate_plan_id	INT UNSIGNED	PK	Surrogate key
lot_id	INT UNSIGNED	FK, ON DELETE CASCADE	Lot
plan_name	VARCHAR(100)	NOT NULL	e.g. 'Weekday daytime'
day_of_week	TINYINT UNSIGNED	NULL = every day; CHECK 1–7	ISO day
start_time / end_time	TIME	NOT NULL	Time band
rate_amount	DECIMAL(8,2)	CHECK ≥ 0	0 = free
rate_unit	ENUM	PER_HOUR, PER_DAY, FLAT, PER_ENTRY	How the amount is charged
free_minutes	SMALLINT UNSIGNED	default 0	Grace period before charging
max_daily	DECIMAL(8,2)	NULL; CHECK ≥ 0	Daily cap
currency	CHAR(3)	default 'GBP'	ISO 4217
effective_from / effective_to	DATE	CHECK to ≥ from	Validity window

`v_occupancy (view)`

Joins each reading to its lot and adds `occupancy_rate` = ROUND(occupied_count / capacity, 4) and `quality_flag` (NEGATIVE if count < 0, OVER_CAPACITY if count > capacity, else OK). Consumers should query this view rather than the raw table.

6. Normalization (3NF)

- **1NF:** every column is atomic; there are no repeating groups. Multiple opening-hour windows, rates or requirements per lot are separate rows, not lists.
- **2NF:** every table has a single-column surrogate primary key, so no non-key attribute can depend on part of a key. The composite business keys (for example `(lot_id, recorded_at)`) fully determine the row's values.
- **3NF:** no non-key attribute depends on another non-key attribute. `capacity` is stored once on `parking_lot`, not repeated on each of the 35,501 readings; requirement names and descriptions live in `requirement_type`, not in `lot_requirement`; occupancy rate and free spaces are derived values and are computed in the view or in queries, not stored.
- **Deliberate choices:** `currency` is stored on each `rate_plan` row (a lot-level currency would also be valid); `source` is stored on each reading so live feeds and the historical dataset can be told apart.

7. Extensibility

- **New special requirements** (EV charging, motorcycle, visitor permit) are new rows in `requirement_type`; no schema change.
- **New lots or campuses** are new `parking_lot` rows; `time_zone` allows lots in different zones. Sections nest through `parent_lot_id`.
- **New data sources** (live sensors, another dataset) use the `source` column and the same unique key.
- **Changing rules over time:** hours, requirements and rates carry `effective_from / effective_to`, so a new schedule is a new row and history is preserved.
- **Special cases covered:** split hours, past-midnight and 24-hour windows, closed days (no row), full and partial closures, open-ended closures, grace periods and free bands.
- **Scale:** the reading table has a unique index on `(lot_id, recorded_at)` for per-lot time-range queries and an index on `recorded_at`; it uses a BIGINT key for long-running live feeds.

8. Data-quality policy and known limitations

- Anomalous readings are **kept and flagged**, not deleted or silently corrected; analysis scripts should filter on `quality_flag = 'OK'` where appropriate.
- The loader uses `INSERT IGNORE` to drop exact-duplicate readings (the unique key rejects them). It would also silently skip other rows that violate that key, so the expected counts (30 lots, 35,501 readings) are checked after each load.
- MySQL cannot enforce `parent_lot_id <> lot_id` as a CHECK on a foreign-key column, so the loader/application must not make a lot its own parent.
- Unique keys that include a nullable column (for example `applies_from` in `lot_requirement`) do not stop duplicates when that column is NULL, because MySQL treats NULLs as distinct.
- Overlapping hours, rate bands or requirement windows for the same lot are not prevented by constraints; the application layer must validate them.
- Hours, closures, requirements and rates have no source data yet (see section 1).

9. Example queries

1. Current availability of every lot (latest reading per lot)

```
SELECT v.lot_code, v.capacity, v.occupied_count,
       v.capacity - v.occupied_count AS free_spaces
FROM v_occupancy v
WHERE v.quality_flag = 'OK'
  AND v.recorded_at = (SELECT MAX(r.recorded_at)
                       FROM occupancy_reading r
                       WHERE r.lot_id = v.lot_id)
ORDER BY free_spaces DESC;
```

2. Average occupancy by hour of day for one lot (website chart / analysis scripts)

```
SELECT HOUR(recorded_at) AS hr, ROUND(AVG(occupancy_rate), 3) AS avg_rate
FROM v_occupancy
WHERE lot_code = 'BHMBCCMKT01' AND quality_flag = 'OK'
GROUP BY HOUR(recorded_at)
ORDER BY hr;
```

3. Data-quality summary

```
SELECT quality_flag, COUNT(*) FROM v_occupancy GROUP BY quality_flag;
```

4. Lots open at a given moment and not fully closed (simple daytime case; overnight and 24 h windows need extra handling)

```
SET @t = '2026-10-12 10:30:00';
SELECT l.lot_code
FROM parking_lot l
JOIN lot_hours h
  ON h.lot_id = l.lot_id
 AND h.day_of_week = WEEKDAY(@t) + 1
 AND TIME(@t) >= h.open_time AND TIME(@t) < h.close_time
 AND h.effective_from <= DATE(@t)
 AND (h.effective_to IS NULL OR h.effective_to >= DATE(@t))
WHERE NOT EXISTS (SELECT 1 FROM lot_closure c
                  WHERE c.lot_id = l.lot_id AND c.closure_type = 'FULL'
                    AND @t >= c.starts_at
                    AND (c.ends_at IS NULL OR @t < c.ends_at));
```

5. Free-parking bands (for the user-facing output)

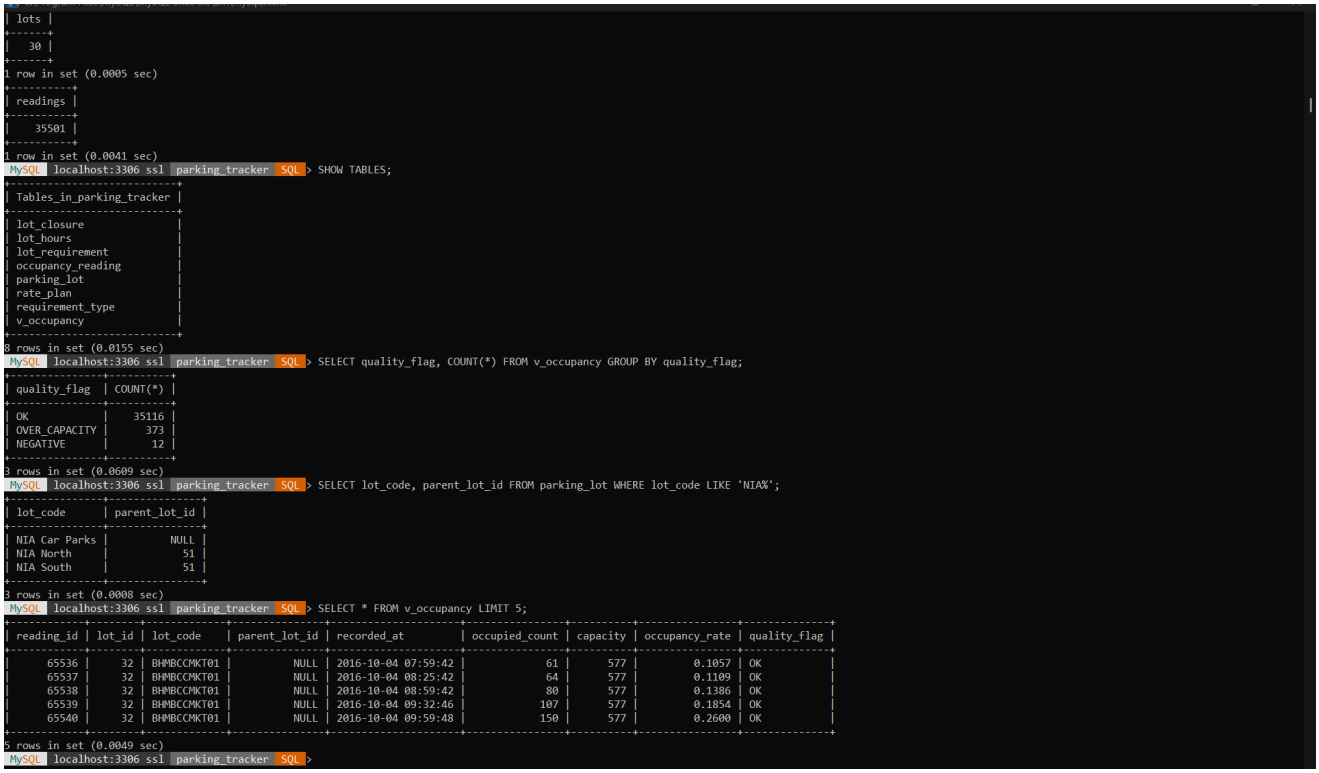
```
SELECT l.lot_code, p.plan_name, p.day_of_week, p.start_time, p.end_time
FROM rate_plan p JOIN parking_lot l ON l.lot_id = p.lot_id
WHERE p.rate_amount = 0;
```

6. Lots with a given special requirement

```
SELECT l.lot_code, t.name, q.spaces_count
FROM lot_requirement q
JOIN parking_lot l       ON l.lot_id = q.lot_id
JOIN requirement_type t  ON t.requirement_type_id = q.requirement_type_id
WHERE t.code = 'DISABILITY_PERMIT'
  AND q.effective_to IS NULL;
```

10. Implementation check in MySQL 8.0.46

The schema was created and the UCI data loaded through MySQL Shell. The screenshot shows the eight objects (seven tables and the view), the load counts (30 lots, 35,501 readings), the quality-flag totals (OK 35,116; OVER_CAPACITY 373; NEGATIVE 12), the NIA hierarchy and sample view rows. The counts match the figures computed independently from the CSV.



```
| lots |
+------+
|   30 |
+------+
1 row in set (0.0005 sec)
+----------+
| readings |
+----------+
|    35501 |
+----------+
1 row in set (0.0041 sec)
MySQL  localhost:3306 ssl  parking_tracker  SQL > SHOW TABLES;
+---------------------------+
| Tables_in_parking_tracker |
+---------------------------+
| lot_closure               |
| lot_hours                 |
| lot_requirement           |
| occupancy_reading         |
| parking_lot               |
| rate_plan                 |
| requirement_type          |
| v_occupancy               |
+---------------------------+
8 rows in set (0.0155 sec)
MySQL  localhost:3306 ssl  parking_tracker  SQL > SELECT quality_flag, COUNT(*) FROM v_occupancy GROUP BY quality_flag;
+--------------+----------+
| quality_flag | COUNT(*) |
+--------------+----------+
| OK           |    35116 |
| OVER_CAPACITY |      373 |
| NEGATIVE     |       12 |
+--------------+----------+
3 rows in set (0.0609 sec)
MySQL  localhost:3306 ssl  parking_tracker  SQL > SELECT lot_code, parent_lot_id FROM parking_lot WHERE lot_code LIKE 'NIA%';
+--------------+--------------+
| lot_code     | parent_lot_id |
+--------------+--------------+
| NIA Car Parks |         NULL |
| NIA North    |           51 |
| NIA South    |           51 |
+--------------+--------------+
3 rows in set (0.0008 sec)
MySQL  localhost:3306 ssl  parking_tracker  SQL > SELECT * FROM v_occupancy LIMIT 5;
+------------+--------+-------------+---------------+---------------------+----------------+----------+----------------+--------------+
| reading_id | lot_id | lot_code    | parent_lot_id | recorded_at         | occupied_count | capacity | occupancy_rate | quality_flag |
+------------+--------+-------------+---------------+---------------------+----------------+----------+----------------+--------------+
|      65536 |     32 | BHMBCCMKT01 |          NULL | 2016-10-04 07:59:42 |             61 |      577 |         0.1057 | OK           |
|      65537 |     32 | BHMBCCMKT01 |          NULL | 2016-10-04 08:25:42 |             64 |      577 |         0.1109 | OK           |
|      65538 |     32 | BHMBCCMKT01 |          NULL | 2016-10-04 08:59:42 |             80 |      577 |         0.1386 | OK           |
|      65539 |     32 | BHMBCCMKT01 |          NULL | 2016-10-04 09:32:46 |            107 |      577 |         0.1854 | OK           |
|      65540 |     32 | BHMBCCMKT01 |          NULL | 2016-10-04 09:59:48 |            150 |      577 |         0.2600 | OK           |
+------------+--------+-------------+---------------+---------------------+----------------+----------+----------------+--------------+
5 rows in set (0.0049 sec)
MySQL  localhost:3306 ssl  parking_tracker  SQL >
```

11. Interface with the rest of the team

- **Python / I/O:** load the CSV into `parking_lot` then `occupancy_reading` and refactor the existing analysis scripts to read from `v_occupancy`. `load_uci.sql` is the reference result; the Python loader should reproduce 30 lots and 35,501 readings. Note: the repository's `parksmart.db` appears to be a different (SQLite) store; the team needs to agree on MySQL as the single database.
- **Website:** current availability and hourly patterns come from queries 1 and 2; project status can quote the counts in section 10.
- **User-facing output:** hours, closures, requirements and free hours/rates are supported by `lot_hours`, `lot_closure`, `lot_requirement` and `rate_plan` (queries 4–6); the team must supply the data.